

Lab Assignment #2
Schedule: Week 5
1. To familiarize with the implementation of merge sort and quicksort 2. To familiarize with numeric order, alphabetical order and lexicographical order

Submission guideline (Online judge)

1. Name your program(s) according to the question (e.g. lab2-q1.c)
2. Write your programs under "Desktop/mywork/lab2"
3. Duplicate your programs before submission
4. Submit your programs under "Desktop/submission/lab2"
5. Check the submission folder for your grading report (Be patience, auto grading need some time)
6. Resubmit your work if you cannot receive full score
7. Visit "Other Locations/Computer/2100share/lab2" in the VM to download the starting programs
8. Version of the gcc compiler in the online judge: C99
9. Deadline: 23:59, 8th October, 2024 (Tuesday)
10. Late penalty is 30%.

Grading scheme (CSCI2100B)

1. Total score is 100%
2. The weighting of each question is marked on the question
3. Bonus score is at most 13%

Requirements for ESTR2102B: Non-zero scores for the extra question

Useful commands

1. Compile your program

```
gcc -o prog labX-q1.c labX-q1-tc1.c
gcc -o prog labX-q3.c
```
2. Run the program with arguments

```
./prog inputfile.txt outputfile.txt
```

Question 1: Merge Sort (40%)

You are asked to write a program to perform merge sort on an unsorted array of integers in descending order. If there are odd numbers in a certain iteration, one more number is divided to the left hand side of the array. Output the whole array after each conquer. A valid conquer refers to at least 2 numbers in the array. You can safely assume that there are no duplicate numbers in the array.

Your program should read the input from the file, and output the answer to another file. The first argument is the input file name, while the second argument is the output file name. Name your program as "lab2-q1.c".

Input file: First line, n, an integer describes the length of the array. Second line, n distinct integers, the unsorted array.

Output file: k lines, k is the number of conquer. Each line should have exactly n numbers separated by a space, indicating the result of the conquer in merge sort.

Sample Input:

```
5
12 16 18 11 17
```

Sample Output:

```
16 12 18 11 17
18 16 12 11 17
18 16 12 17 11
18 17 16 12 11
```

Question 2: Insertion Sort (30%)

You are asked to write a program to perform insertion sort on an array of words in the ascending alphabetical order. The alphabetical order is defined as (i) capital letter is smaller than its small letter, and (ii) 'A' is the smallest and 'z' is the largest (i.e. 'A' < 'a' < 'B' < 'b' < ... < 'Z' < 'z'). Output the whole array after each iteration. You can safely assume that all strings are distinct and contain no spaces, and the length of a word is at most 100.

Your program should read the input from the file, and output the answer to another file. The first argument is the input file name, while the second argument is the output file name. Name your program as "lab2-q2.c".

Input file: First line, n, an integer describes the length of the array. Second line, n distinct words, the unsorted array.

Output file: n-1 lines, each line should have exactly n words separated by a space, indicating the result of each iteration in selection sort.

Sample Input:

4

Apple Orange Banana Pear

Sample Output:

Apple Orange Banana Pear

Apple Banana Orange Pear

Apple Banana Orange Pear

Question 3: Quicksort (30%)

You are asked to write a program to perform quicksort on an array of strings in the ascending lexicographical order (i.e. ASCII code). You should select the pivot as the median of the first, middle and last element. In addition, you need to hide the pivot at the first position of the array in its iteration. Consider l , c , r are the indices of the three candidates. After hiding the pivot, the median is stored at l , the largest one is stored at r , while the smallest one is stored at c . Output the whole array after each partition. A valid partition refers to the partition of 3 or more numbers, otherwise, we are unable to select the median-of-three as pivot. You can safely assume that all strings are distinct and contain no spaces, and the length of a string is at most 100.

Your program should read the input from the file, and output the answer to another file. The first argument is the input file name, while the second argument is the output file name. Name your program as "lab2-q3.c".

Input file: First line, n , an integer describes the length of the array. Second line, n distinct strings, the unsorted array.

Output file: k lines, k is the number of partition steps. Each line should have exactly n strings separated by a space, indicating the result of the partition. Then, print an empty line. Then, output exactly n strings separated by a space, the result of quicksort.

Sample Input:

4

Apple Orange Banana Pear

Sample Output:

Banana Apple Orange Pear

Apple Banana Orange Pear

Question 4: Generic Data Type in C? (Bonus 13%)

A generic type means parameterized types. Indeed, C does not support generic types. However, we can use void pointers to pretend generic types. You are going to implement the content for the `genericsort()` function prototype, sorting algorithm for general data type, defined below. The header file "lab2-q4.h" and the description of the function are as follows.

Your program should not contain `main()`. Name your program as "lab2-q4.c".

```
// lab2-q4.h
// Cannot modify this file

// sort the array according to the order specified by comparefunc
// positive return value from comparefunc means swapping is needed
// datatype size is the size of the data
// e.g. datatype size = 4 when the input array is an int array
// arraylength is the number of elements to be sorted in array
// after calling the function, the input array becomes sorted
// you can use any sorting algorithm for this problem
void genericsort(void *array, int datatype size, int arraylength,
int (*comparefunc)(void*,void*));
```

Sample Test Case (lab2-q4-tc1.c):

```
// This file is named as lab2-q4-tc1.c
#include "lab2-q4.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>

int comparefunction(char *A, char *B) {
    return (*B) - (*A);
}

int main(int argc, char *argv[]){
    FILE *fout = fopen(argv[1], "w");
    char word[10] = "AaBbCc";
    char ans[10] = "ABCabc";
    int dtsize = sizeof(char);
    int arraylength = 6;

    // Assume your program is correct
    int isCorrect = 1;

    // calling genericsort()
    genericsort(word, dtsize, arraylength, comparefunction);

    // Test your answer
    if (strcmp(word, ans) != 0) isCorrect = 0;

    // Output test result
    if (isCorrect) fprintf(fout, "Sort: Correct\n");
    else fprintf(fout, "Sort: Wrong Answer\n");
    return 0;
}
```